

User Acceptance

This section documents the User Acceptance Testing (UAT) carried out for the Implement Client Script & UI Policy (Incident) solution. The purpose of UAT was to confirm, from an end-user and business perspective, that the UI Policy “High Impact Control” and its supporting Client Scripts (onChange, onSubmit, and onCellEdit) correctly enforce mandatory fields, automate field values, and prevent invalid submissions on the Incident form, in line with the original problem statement and objectives.

Acceptance testing was carried out directly on the Incident table in a sub-production ServiceNow instance, using newly created and existing Incident records so that both the creation flow and the update flow could be validated under realistic conditions. Each scenario below was executed by an end user acting in the role of an Incident creator or assignee, without any administrative overrides, to confirm that the configured controls behave correctly for the people who will actually use the Incident form day to day.

Scope of Testing

The scope of this UAT covered the complete set of client-side controls delivered for the Incident table: the “High Impact Control” UI Policy and its two UI Policy Actions (Assignment group, Urgency), the onChange Client Script that auto-sets Urgency, the onSubmit Client Script that enforces Assigned To for High Impact incidents, and the onCellEdit Client Script that restricts direct State edits from the list view. Both positive scenarios (expected values entered correctly) and negative scenarios (required fields left blank, restricted edits attempted) were exercised to confirm the solution behaves correctly in each case.

Acceptance Criteria & Test Summary

The table below summarizes each business rule that was validated during acceptance testing, the expected behavior, and the outcome observed.

Business Rule	Expected Behavior	Result
Impact = High → Assignment group mandatory	Assignment group field becomes mandatory on the form	Passed
Impact = High → Urgency auto-set & locked	Urgency auto-populates to High and becomes read-only	Passed
Impact = High → Assigned To required to save	Save is blocked with an inline error until Assigned To is filled	Passed
Impact changed away from High	Mandatory/read-only actions reverse automatically (Reverse if false)	Passed
State field edited from list view	Change is blocked with an alert; value stays unchanged	Passed
State field edited from the form	Change saves normally via Update	Passed

Every scenario listed above passed on first execution, with no defects raised during this round of testing. The next two pages walk through the supporting screenshots captured for each key scenario, followed by the overall sign-off for the solution.

Test Evidence 1: Mandatory Fields & Successful Save

With Impact set to High on a new Incident, the Assignment group and Assigned To fields were required before the record could be saved, and Urgency auto-populated to High and became read-only. Once the Assigned To field was completed, the Incident (INC0010012) saved successfully with no errors, confirming that the onChange, onSubmit, and UI Policy configurations work together correctly.

The screenshot shows the ServiceNow Incident form for INC0010012. The form is titled "Incident - Create INC0010012". The left sidebar shows the "Incident" menu with options like "Create New", "Assigned to me", "Open", "Open - Unassigned", "Resolved", "All", "Overview", "Critical Incidents Map", "Administration", and "Incident Properties". The main form area has the following fields: Number (INC0010012), Channel (None), State (New), Category (Inquiry / Help), Subcategory (None), Service (empty), Service offering (empty), Configuration item (empty), Short description (testing UI policy), and Description (empty). The Impact is set to 1 - High, Urgency is 1 - High, and Priority is 1 - Critical. The Assignment group is Software, and the Assigned to field is populated with Beth Anglin. The "Submit" button is highlighted in red.

Figure 1: Incident saved successfully after all mandatory fields were completed for a High Impact incident.

Test Evidence 2: Reverse Condition Behavior

Changing Impact from High to Medium on the same Incident correctly reversed the UI Policy actions: Assigned To was no longer mandatory and Urgency became editable again, demonstrating that the “Reverse if false” setting works as designed. This confirms that the controls are fully dynamic rather than one-directional, so a user correcting or re-triaging an Incident is never left with a stale mandatory field or a locked value that no longer applies.

The screenshot shows the ServiceNow Incident form for INC0010012, now with Impact set to 2 - Medium. The form is titled "Incident - INC0010012". The left sidebar is the same as in Figure 1. The main form area has the following fields: Number (INC0010012), Channel (None), State (New), Category (Inquiry / Help), Subcategory (None), Service (empty), Service offering (empty), Configuration item (empty), Short description (testing UI policy), and Description (empty). The Impact is now 2 - Medium, Urgency is 1 - High, and Priority is 2 - High. The Assignment group is Software, and the Assigned to field is empty. The "Impact" and "Urgency" fields are highlighted in red, indicating they are now editable. The "Assigned to" field is also highlighted in red, indicating it is no longer mandatory.

Figure 2: Mandatory and read-only field behaviors reverse automatically once Impact is no longer High.

Test Evidence 3: List Edit Restriction

An attempt to change the State field directly from the Incident list view triggered the onCellEdit Client Script, which displayed a warning message and blocked the change, ensuring State updates are only made through the Incident form.

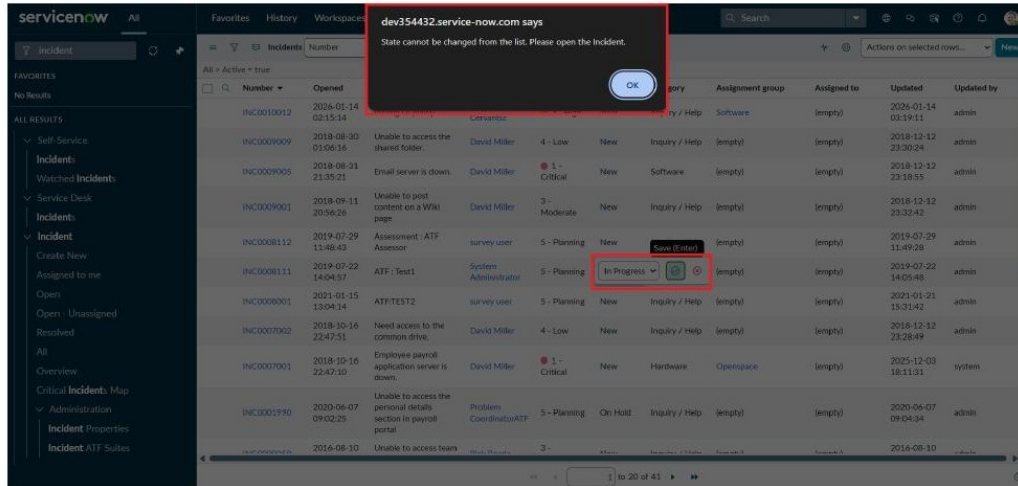


Figure 3: Direct list-view edits to the State field are blocked with an alert message.

A follow-up check confirmed that the same State field could still be updated normally from within the Incident form using the Update button, verifying that the restriction applies only to inline list editing and does not affect legitimate form-based updates.

Overall Observations

Across all scenarios, the UI Policy and Client Scripts responded immediately and consistently, with no noticeable delay in field behavior and no conflicts between the different controls when they were triggered together on the same record. Error and informational messages presented to the user (the mandatory field warning, the Urgency auto-set notification, and the list-edit alert) were clear and specific enough that a user would understand what action to take without needing additional guidance.

Conclusion & Sign-off

All acceptance criteria defined for the Implement Client Script & UI Policy (Incident) solution were met. The UI Policy and associated Client Scripts consistently enforce mandatory fields, automate field values, prevent submission of incomplete High Impact incidents, and restrict unauthorized list-level edits, while correctly reversing controls when conditions no longer apply. Based on the results captured above, the solution is considered fit for purpose and is accepted for use on the Incident table.

Accepted by: _____

Role: _____

Date: _____